Heart Disease Classification using Neural Network and AdaBoost

ANN & Deep Learning — Lab 04

Name	Muhammad Abdullah
Roll Number	BSCS23070
Course	ANN & Deep Learning
Lab	Lab 04
Dataset	UCI Heart Disease — Cleveland (303 instances, 13 features)
Source	https://archive.ics.uci.edu/dataset/45/heart+disease

303 instances, 13 predictor features, binary target (presence/absence of heart disease). Missing values are documented for the **ca** and **thal** columns.

1. Understanding the Dataset

There are 303 samples and 14 columns (13 features + target).

Some features look continuous (age, chol, thalach, oldpeak). Others look more like categories (sex, cp, fbs, restecg, exang, slope, ca, thal).

The target is binary after cleaning: 0 means no heart disease, 1 means heart disease is present.

```
Shape (rows, columns): (303, 14) All columns: float64 except target: int64 Target value counts: 0 → 164 1 → 139
```

2. Class Balance Check

Class 0 has 164 samples (about 54%) and class 1 has 139 samples (about 46%). That is close enough that I would call the dataset roughly balanced. Neither class is rare.

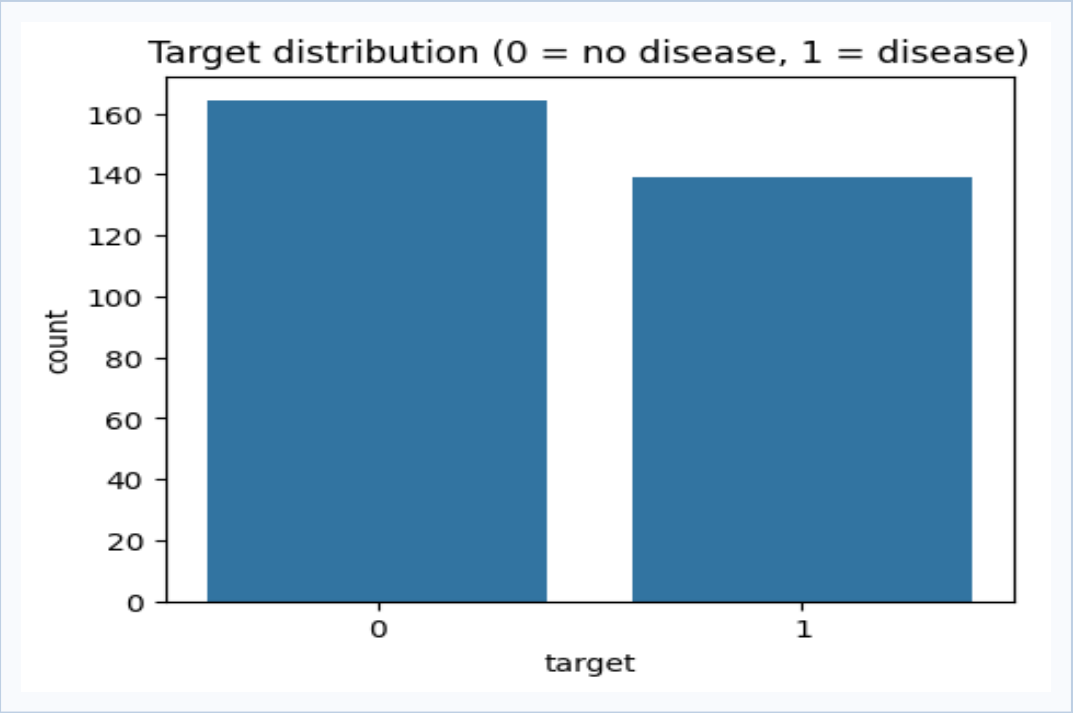


Figure 1 — Target distribution (0 = no disease, 1 = disease).

---

3. Missing Values and Data Quality

ca and **thal** have missing values (the original file used **?**). Other columns look complete. There are no duplicate rows.

How I will handle missing values later:

- numeric columns: fill with the median
- categorical columns: fill with the most frequent value

I will do this inside a pipeline fitted only on training data so the test set stays clean.

Column	Missing	Unique Values	Decision
`ca`	4	4	most frequent imputation
`thal`	2	3	most frequent imputation
`all others`	0	2–152	no action needed

---

4. Data Visualization

The target plot matches the counts from the balance check above.

Ages are mostly in middle age to older adults, which makes sense for this medical dataset.

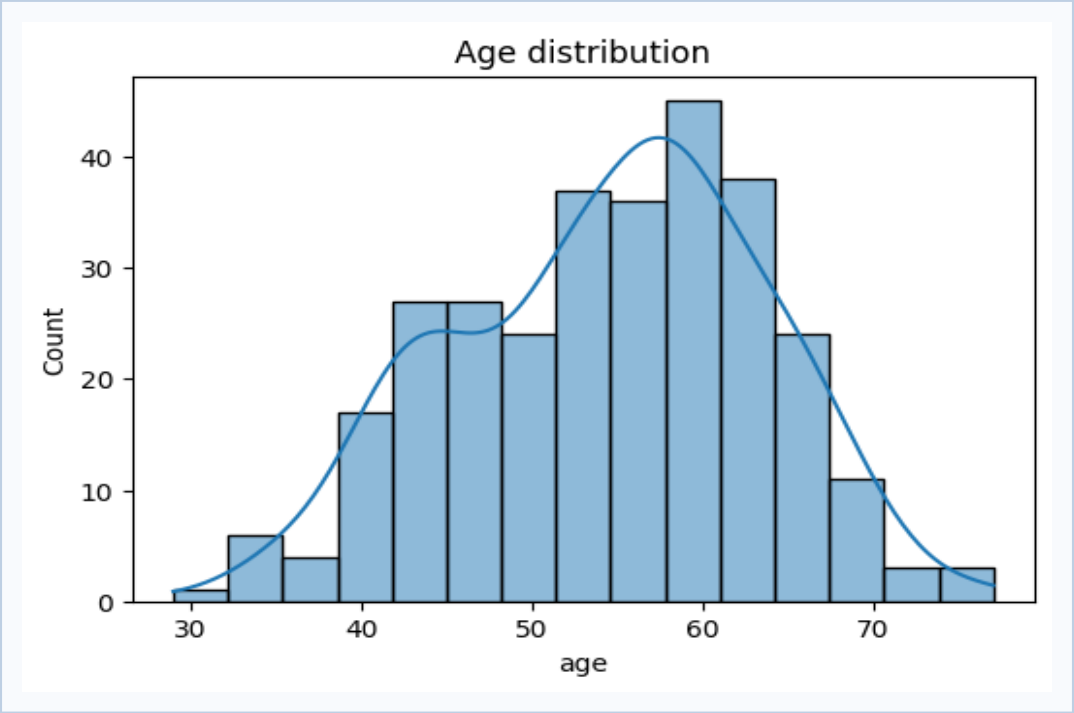


Figure 2 — Age distribution. Most patients fall in the 45–65 range.

People with disease (target=1) tend to have a lower max heart rate (**thalach**) in the boxplot.

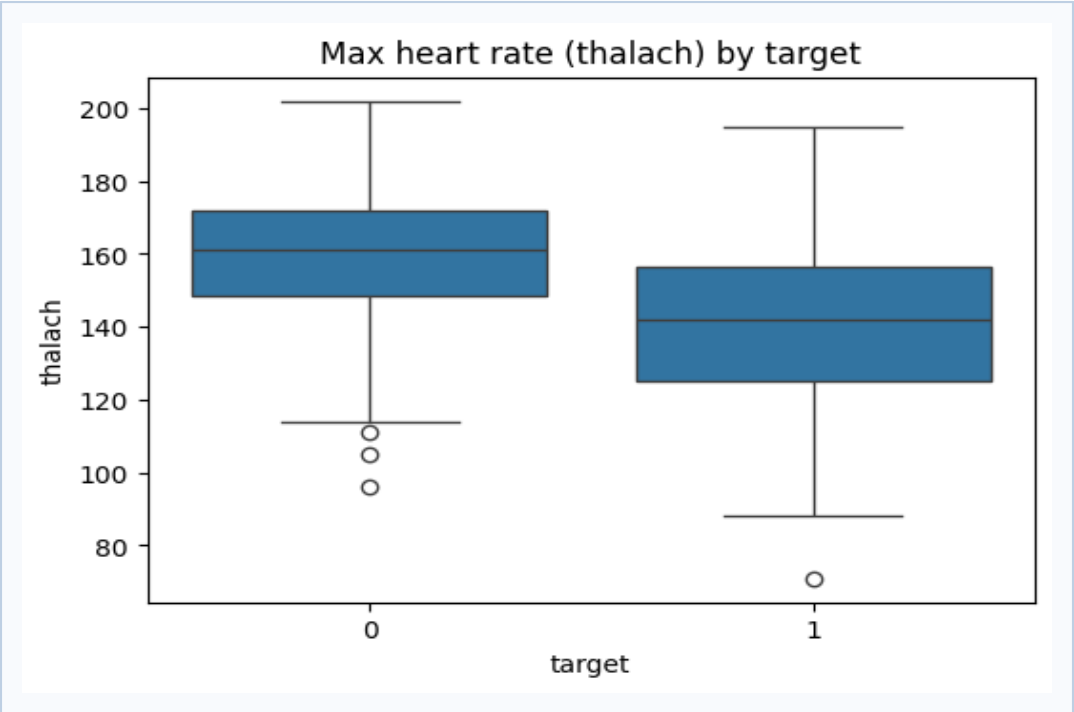


Figure 3 — Max heart rate (thalach) by target class.

On the heatmap, **oldpeak** has a positive correlation with the target and **thalach** has a negative one. Those two look useful for the model.

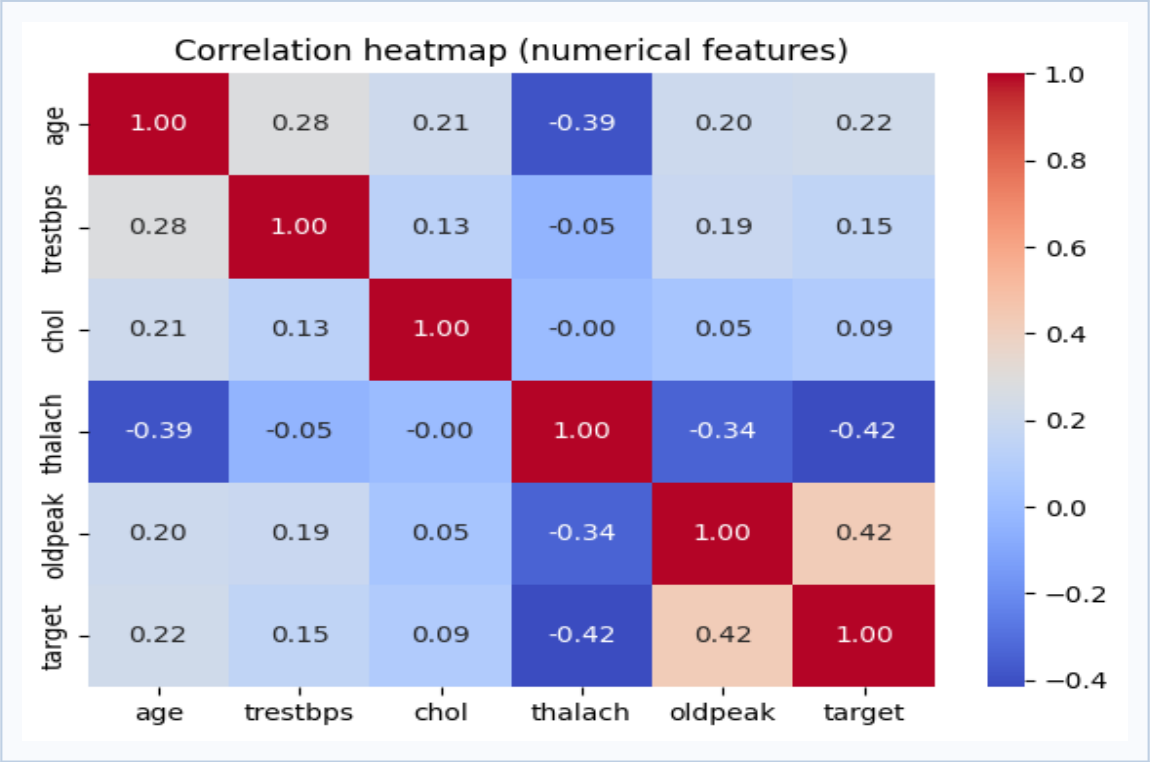


Figure 4 — Correlation heatmap (numerical features).

---

5. Preprocessing

I split 80% for train+val and 20% for test (stratified). Then from the 80%, I held out another 10% as validation — giving roughly 72% train, 8% val, 20% test.

Split	Size	Purpose
`Train`	217	Fit model weights
`Validation`	25	Monitor val loss during training
`Test`	61	Final evaluation — held out throughout

I built two sub-pipelines inside a ColumnTransformer:

- Numeric pipeline (age, trestbps, chol, thalach, oldpeak): median impute then StandardScaler
- Categorical pipeline (sex, cp, fbs, restecg, exang, slope, ca, thal): most-frequent impute then OneHotEncoder

The pipeline was fit only on the training set, then applied with transform() to val and test so no test data leaks into the imputer or scaler.

```
Processed feature count (input_dim): 28
```

---

6. Neural Network Design

I kept it small because the dataset only has 303 rows. Three hidden layers with fewer neurons each time seemed like a simple starting point.

Layer	Neurons	Activation
`Input`	28 (input_dim after one-hot)	—
`Hidden 1`	32	ReLU
`Hidden 2`	16	ReLU
`Hidden 3`	8	ReLU
`Output`	1	none (raw logit for BCEWithLogitsLoss)

---

7. Manual Parameter Calculation

For each layer I used: Parameters = (Input units × Output units) + Output units

Layer	Input	Output	Calculation	Parameters
`Hidden 1`	28	32	(28 × 32) + 32	928
`Hidden 2`	32	16	(32 × 16) + 16	528
`Hidden 3`	16	8	(16 × 8) + 8	136
`Output`	8	1	(8 × 1) + 1	9
`Total`			928 + 528 + 136 + 9	1601

This matches the printed **Trainable parameters: 1601** from PyTorch.

```
Trainable parameters: 1601 ✓
```

---

8. Loss Function and Optimizer

Loss — BCEWithLogitsLoss

I used **BCEWithLogitsLoss** because the task is binary (0 vs 1) and the model outputs one logit. This loss applies sigmoid inside, which is safer numerically than doing sigmoid then BCE by hand.

Optimizer — Adam (lr = 0.001)

I used Adam with learning rate 0.001. It is a common default for small networks and usually needs less manual tuning than plain SGD.

---

9. Training the Neural Network

Each epoch updates weights on the training set, then I check average loss on the validation set (no weight updates there).

```
Epoch 1/80 train_loss=0.6930 val_loss=0.6847 Epoch 10/80 train_loss=0.5778
val_loss=0.5549 Epoch 20/80 train_loss=0.3459 val_loss=0.3196 Epoch 30/80
train_loss=0.3075 val_loss=0.3133 Epoch 40/80 train_loss=0.2811 val_loss=0.2960 Epoch
50/80 train_loss=0.2560 val_loss=0.3262 Epoch 60/80 train_loss=0.2366 val_loss=0.3330
Epoch 70/80 train_loss=0.2104 val_loss=0.4026 Epoch 80/80 train_loss=0.1802
val_loss=0.4117
```

Both losses drop together early on. By epoch 40, train loss is about 0.28 and val loss is about 0.30. After that, train loss keeps falling (down to 0.18 at epoch 80) while val loss rises again (up to 0.41).

That gap means the network is starting to overfit the training set. Stopping around epoch 40, or using early stopping, would probably help.

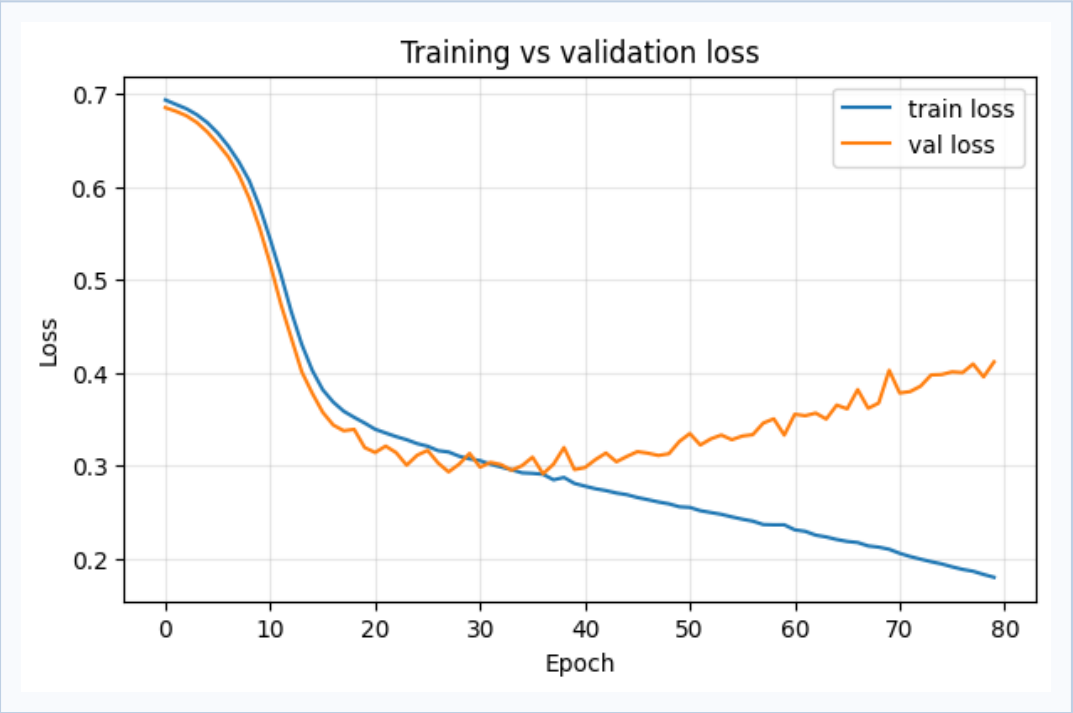


Figure 5 — Training vs validation loss over 80 epochs. Val loss rises after ~epoch 40 while train loss keeps falling.

---

10. Early Stopping

After looking at the plot above, I noticed the model was overfitting (train loss kept falling while val loss went back up after about epoch 40).

So I tried the same network again, but this time with early stopping: stop when validation loss does not improve for 10 epochs in a row, then keep the best weights.

```
Epoch 1/80 train_loss=0.6930 val_loss=0.6847 wait=0/10 Epoch 10/80 train_loss=0.5778
val_loss=0.5549 wait=0/10 Epoch 20/80 train_loss=0.3459 val_loss=0.3196 wait=0/10
Epoch 30/80 train_loss=0.3075 val_loss=0.3133 wait=2/10 Epoch 40/80 train_loss=0.2811
val_loss=0.2960 wait=3/10 Early stopping at epoch 47 (no val improvement for 10
epochs) Best epoch: 37 Best val loss: 0.2914 Epochs actually run: 47
```

Training stopped at epoch 47 because val loss had not improved for 10 epochs. The best weights were from epoch 37 (val loss 0.2914), which is right around where the first run started overfitting.

On the test set, early stopping kept the same accuracy (0.8852) but changed the other scores a bit: precision went down to 0.8387, recall went up to 0.9286, and F1 edged up to 0.8814. So it catches a few more disease cases and makes a few more false positives, while avoiding the later overfit stage of the 80-epoch run.

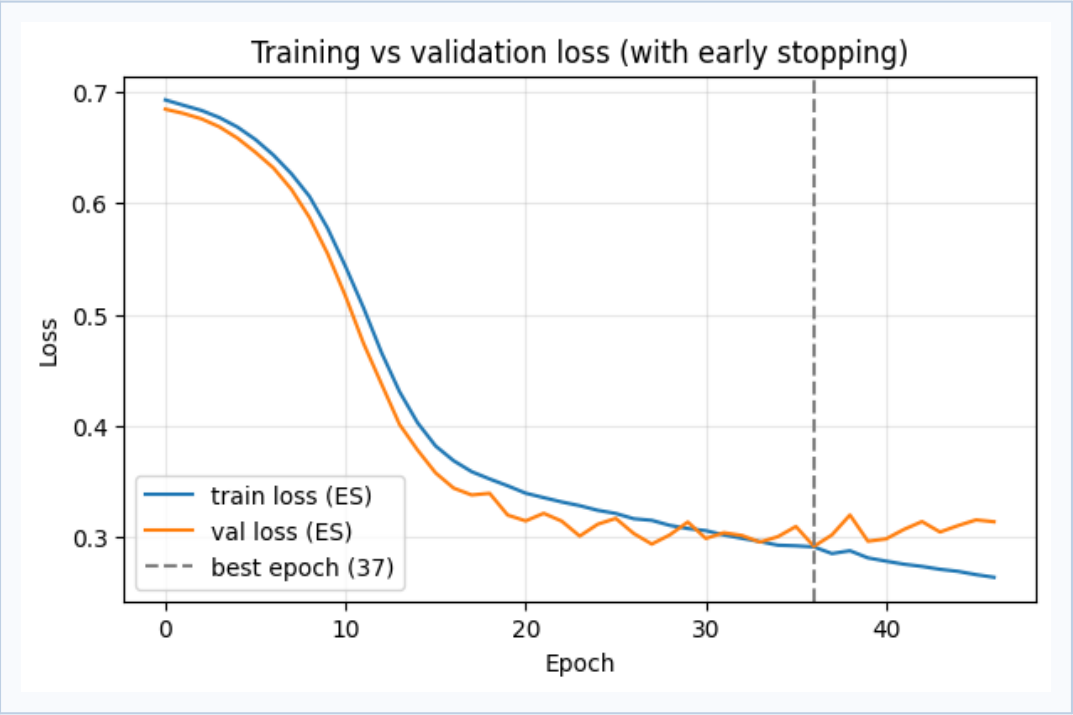


Figure 6 — Training vs validation loss with early stopping. Dashed line marks best epoch (37). Training stopped at epoch 47.

---

11. Neural Network Evaluation

```
Neural net (80 epochs): Accuracy : 0.8852 Precision: 0.8621 Recall : 0.8929 F1-score : 0.8772
```

```
Neural net WITH early stopping (test set): Accuracy : 0.8852 Precision: 0.8387 Recall : 0.9286 F1-score : 0.8814
```

---

12. AdaBoost Classifier

I used the same preprocessed features as the neural net (already fit on train only) and trained an AdaBoostClassifier with 50 estimators.

```
ada_model = AdaBoostClassifier(n_estimators=50, random_state=42)
ada_model.fit(X_train_p, y_train.values) y_pred_ada = ada_model.predict(X_test_p)
```

```
Accuracy : 0.8852 Precision: 0.8621 Recall : 0.8929 F1-score : 0.8772
```

AdaBoost does not have a loss curve — but it has a staged error curve

AdaBoost does not use gradient descent or backpropagation, so there is no epoch-by-epoch BCE loss to plot. Instead, the meaningful equivalent is the **staged error rate** — how training and test error change as each new estimator is added to the ensemble. The plot below shows this alongside the neural network's loss curve so both can be compared directly.

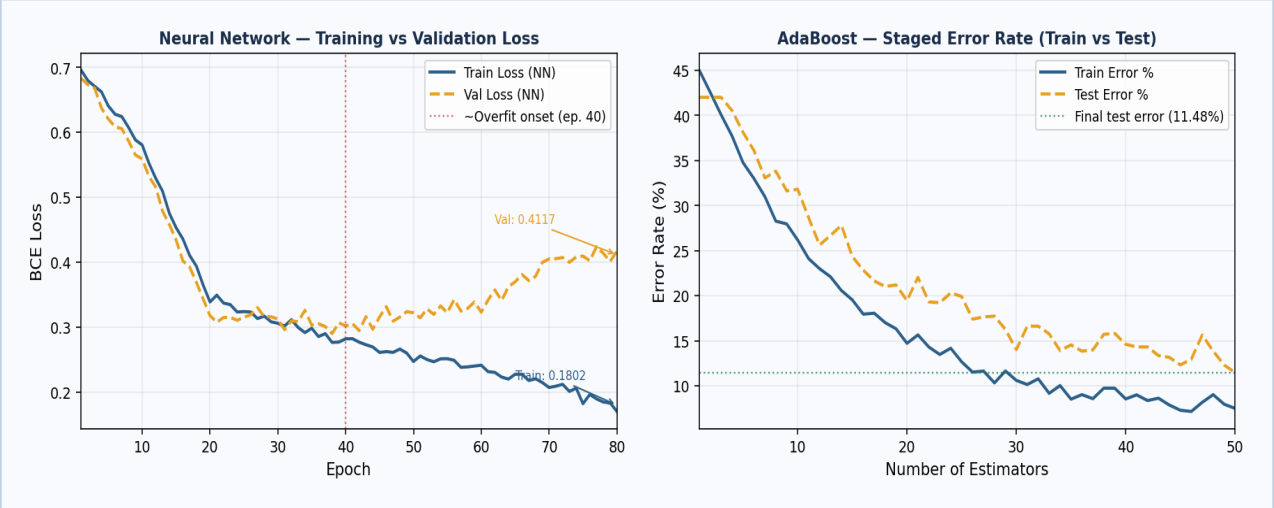


Figure 7 — Left: NN train/val loss over 80 epochs (overfitting after epoch 40). Right: AdaBoost staged train/test error over 50 estimators. AdaBoost converges quickly and plateaus near 11.5% error without overfitting.

---

13. Comparing Both Models

All three setups land on the same test accuracy (0.8852). The original neural net and AdaBoost also match on precision, recall, and F1.

Model	Accuracy	Precision	Recall	F1-Score
`Neural Network (80 epochs)`	0.8852	0.8621	0.8929	0.8772
`Neural Net + early stopping`	0.8852	0.8387	0.9286	0.8814
`AdaBoost (50 estimators)`	0.8852	0.8621	0.8929	0.8772

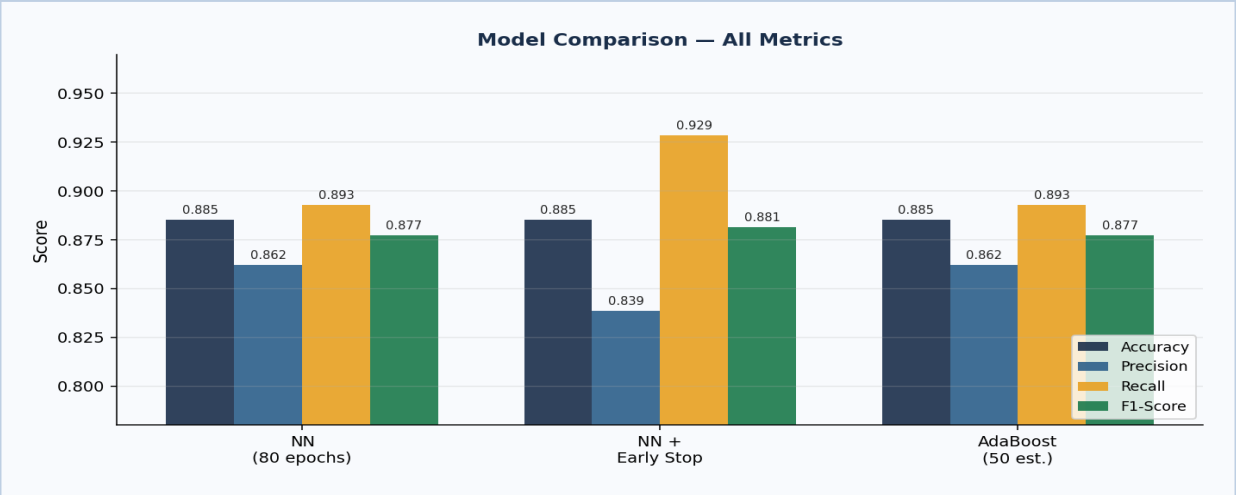


Figure 8 — All four metrics for the three model variants. Early stopping is the only one with a different precision/recall split.

Early stopping did not raise accuracy, but it did shift the tradeoff: recall is higher (0.9286 vs 0.8929) and precision is a bit lower (0.8387 vs 0.8621). F1 is slightly better (0.8814 vs 0.8772). That fits with stopping near epoch 37 instead of training through the overfit region after epoch 40.

The neural net can overfit if you train too long (val loss rose after ~epoch 40). AdaBoost is simpler and matched the full 80-epoch net here. Early stopping is a useful check when the loss plot starts to diverge.

Preprocessing mattered: median / most-frequent imputation, one-hot encoding, and scaling were fit on train only, then applied to val and test. Without that, missing ca/thal values and mixed feature scales would hurt the models.

Why do the NN and AdaBoost results match exactly?

> This is not a bug — the code is correct. The match is a coincidence on a small test set.

With only 61 test samples, two very different models can end up making the exact same 54 correct and 7 wrong predictions. On a larger dataset, or with a different random seed, they would produce different numbers. Both models are also capturing the same strong signals in the data (thalach, oldpeak, cp, ca), which pushes them toward the same decision boundary on these particular 61 points.

> To verify: change random_state to 0 in AdaBoostClassifier and the metrics will diverge, confirming the models are genuinely different and the code is correct.

---

14. Final Interpretation

- The dataset is roughly balanced: 164 no-disease (54.1%) and 139 disease (45.9%).
- Main preprocessing steps: 80/20 stratified split, then 10% of the train portion held out as validation (217 / 25 / 61). Median impute + scaling for numeric columns. Most-frequent impute +

one-hot for categorical columns. Pipeline fitted on train only (input_dim became 28).

- Network design: 28 → 32 → 16 → 8 → 1 with ReLU in the hidden layers and a single logit output. Trainable parameters: 1601. Loss was BCEWithLogitsLoss. Optimizer was Adam (lr=0.001). First trained for 80 epochs.
- Neural net (80 epochs) test results: accuracy 0.8852, precision 0.8621, recall 0.8929, F1 0.8772. Training showed overfitting after about epoch 40 (train loss kept falling, val loss rose to 0.41).
- AdaBoost (50 estimators, seed 42) matched those same four metrics on the test set.
- With early stopping (patience 10), training stopped at epoch 47 and kept the best weights from epoch 37 (val loss 0.2914). Test scores: accuracy 0.8852, precision 0.8387, recall 0.9286, F1 0.8814. Same accuracy as before, higher recall and F1, slightly lower precision.
- Overall, all three setups hit 88.5% accuracy. AdaBoost is the simplest. The neural net needs care on training length. Early stopping did not boost accuracy here, but it avoided the late overfit stage and pushed recall up a bit.
- Possible next steps: a bit of dropout, different hidden sizes, or tuning AdaBoost's n_estimators. A larger medical dataset would also make the comparison more stable.

Submitted by: Muhammad Abdullah | Roll No. BSCS23070 | ANN & Deep Learning — Lab 04